Jeu d'Aventure

TP en temps libre d'Analyse, Conception et Programmation Orientée Objet

Ensimag 2A MMXXVI

Dernière modification : 17 septembre 2026 12h

L'objectif de ce TP est de concevoir et développer en `Java` un jeu d'aventure, avec différentes salles et des portes qui permettent de passer d'une salle à l'autre.

La première partie décrit les exigences fonctionnelles du jeu. La deuxième décrit l'architecture MVC attendue. La troisième propose une partie en bonus et contraint l'utilisation d'IA dans le projet. Les quatrième et dernière partie décrivent les livrables attendues et les critères d'évaluation.

Ce TP vous permettra d'aborder au fur et à mesure les aspects fondamentaux de la programmation orientée objet : encapsulation, délégation, héritage, abstraction et utilisation des collections `Java` ; ainsi que les techniques d'analyse et conception objet de logiciel vues en cours.

Le travail est prévu pour des groupes de quatre étudiant.e.s (ou, à défaut, trois). Attention la charge de travail est importante. Bien entendu, vos enseignant.e.s sont là pour vous aider et répondre à vos questions (mais cela exige que vous vous y preniez assez tôt...).

Le projet a deux rendus : un rendu intermédiaire le **vendredi 23 octobre 2026, 20h** ; et un rendu final le **jeudi 03 décembre 2026, 20h**.

Bon travail à tou.te.s !

1 Première partie : développement du jeu

On cherche à concevoir et implémenter un jeu d'aventure. Un personnage se retrouve dans une maison qu'il ne connaît pas, avec différentes salles et des portes qui permettent de passer d'une salle à l'autre. Certaines portes sont fermées à clé.

Au début de l'aventure, le personnage est plein de vie (même s'il a un sac vide). Ceci peut changer au fur et à mesure que le jeu avance.

Sur son chemin le personnage peut ramasser divers objets, par exemple des clés, de la nourriture, et des armes, qu'il peut stocker dans son sac (dont la capacité est limitée). Des monstres plus ou moins hostiles peuvent être rencontrés et combattus dans les salles.

Le but du jeu est double : découvrir un trésor caché et s'échapper de la maison en vie. Dans l'Annexe A vous trouverez un exemple de format de données pour simuler la maison.

1.1 Fonctionnalités

1.1.1 Combat

Le personnage peut combattre des monstres avec ses armes et perdre des points de vie pendant un combat (le nombre de points de vie doit être configurable). Le personnage peut récupérer des points de vie en consommant de la nourriture.

Il existe différentes stratégies de combat entre le personnage et les monstres plus au moins complexes à implémenter. Par exemple, en fonction de l'arme que le personnage ou le monstre utilise, les attaques d'un causent des dégâts à l'autre lorsque l'attaqué est touché, et chacun peut attaquer à son tour. Décrivez dans votre rapport quelle stratégie vous avez implémenté.

1.1.2 Choix de la maison

Le jeu inclut à minima une maison qui est chargée au début avec des salles (éventuellement dans plusieurs étages), de la nourriture, des armes et des monstres à combattre.

Dans un deuxième temps, ajoutez la possibilité de choisir parmi plusieurs maisons de quelle maison le personnage doit s'échapper.

1.1.3 Des allié.e.s (mais de qui ?)

En plus des objets, le personnage peut rencontrer d'autres personnages semblant sympathiques pendant son parcours. Ils lui aideront pendant les combats sanglants avec leurs propres points de vie (qui sont en même nombre que le personnage principal).

Les monstres peuvent attaquer soit le personnage principal soit un des personnages secondaires du groupe, mais jamais plus qu'un en même temps.

Attention, un personnage qui semblait sympathique peut très bien montrer sa véritable intention lorsqu'un monstre est rencontré : il peut être en effet un allié du monstre, et doit être tué aussi !

1.1.4 Configurations

Leu jeu doit inclure la possibilité de configurer différents paramètres tels quels :

— le nombre de points de vie du personnage principal du jeu ;
— le nombre de points de vie des monstres (qui auront tous le même nombre de points de vie) ;
— la capacité de stockage du sac du personnage principal (en nombre d'objets) ;
— le nombre de personnages secondaires que l'on peut rencontrer dans la maison ;
— d'autres paramètres que vous trouvez utiles.

1.2 Interface du jeu

Pour cette première partie du jeu, vous pouvez vous contenter de développer une interface textuelle (donc non graphique), que vous intégrerez au rendu intermédiaire. En bonus, une interface graphique pourra être développée pour le rendu final (détails en section 2). Bien entendu, si vous le souhaitez, rien ne vous empêche de faire une interface graphique dès le rendu intermédiaire.

2 Deuxième partie : architecture MVC

Vous proposerez une architecture MVC (modèle-vue-contrôleur) pour votre application. Vous devrez effectuer une restructuration de votre code en accord avec cette architecture.

3 Troisième partie : Bonus et IA

À l'aide d'une IA, vous pouvez générer une interface graphique pour votre jeu partir de l'architecture MVC proposée dans la partie 2. Dans votre rapport, vous préciserez l'IA employée et vous montrerez différents prompts permettant d'arriver à votre solution. Vous préciserez si vous avez eu besoin de beaucoup modifier le code généré, et à quels endroits, pour que cette interface graphique fonctionne bien. Vous pouvez aussi bien entendu créer une interface graphique sans utiliser une IA.

Toute autre utilisation de l'IA pour réaliser ce TP, pour le code ou les rapports, est interdite et à proscrire dans le module.

4 Livrables attendus

Chaque rendu sera déposé sous `Moodle` sous la forme d'un seul archive `tar.gz`.

4.1 Rendu Intermédiaire

Pour rappel, pour le rendu intermédiaire une interface textuelle est suffisante pour votre application. Le rendu intermédiaire ne contient que la première partie du sujet (développement du jeu).
L'archive `tar.gz` rendu doit contenir :

1. le **code source** de votre application ;
2. un **rapport** au format `pdf` de 4 pages maximum expliquant et justifiant vos choix de conception, l'utilisation à bon escient des classes et des méthodes les plus adaptées, la stratégie de combat, les limites de votre jeu, etc. Vous décrirez également les principaux tests effectués et résultats obtenus ;
3. un **documentation utilisateur** de 3 pages maximum, décrivant votre programme du point de vue de l'utilisateur. Éventuellement (si applicable, et si ce n'est pas évident), il doit décrire les interfaces graphiques ;
4. un fichier **Makefile** fonctionnel. Il doit décrire comment compiler et exécuter le programme. Ensuite les principales commandes disponibles pour utiliser votre programme. Ce document sera essentiel pour l'évaluation ;
5. un **README** expliquant les différents dossiers et les principaux fichiers de l'archive `tar.gz`.

4.2 Rendu Final

Le rendu final contient les parties 2 et 3 du sujet (l'architecture MVC et l'interface graphique en bonnus).
L'archive `tar.gz` doit contenir :

1. le **code source** de votre application mis à jour suite à l'application de l'architecture MVC ;
2. le **rapport** mis à jour, toujours au format `pdf`, maintenant avec 8 pages maximum. Vous ajouterez au rapport précédent la description de l'architecture MVC appliqué à votre application, quelles parties du code ont changé lors de l'application du patron MVC, le diagramme de classes logicielles et, si vous avez utilisé une IA pour les interfaces graphiques, les prompts permettant d'arriver à votre solution ;
3. un fichier **Makefile** fonctionnel et mis à jour ;
4. le **README** mis à jour.

5 Critères d'évaluation

Le projet rendu devra répondre aux spécifications des différentes parties ci-dessus. Si toutes les exigences ne sont pas prises en compte, bien le spécifier dans le rapport dans une section **Limites du Projet**.

Chaque rendu évalue une partie du module (POO et Analyse/Conception). Le rendu intermédiaire évalue la première partie du **module** seulement (correspondant à la partie 1 du sujet), et le rendu final n'évalue que la deuxième partie du **module** (correspondant aux parties 2 et 3 du sujet).

5.1 Rendu Intermédiaire

Ce rendu se fait à la moitié du module, pour évaluer les notions de la programmation orientée à objets. Dans ce sens, au-delà des exigences fonctionnelles décrites précédemment, nous allons évaluer l'ensemble de notions vues en cours. Plus précisément :

1. Fonctionnalités
   - **Rendu :** Les éléments permettant de compiler et d'exécuter facilement le projet sont présents, ainsi que le rapport.
   - **Exigences :** La réalisation répond au cahier des charges.
2. Compétences
   - **Qualité du code :** *coding style*, indentation, noms de variables, commentaires, exceptions, etc...

— **Encapsulation :** Les principes sont bien respectés : masquage des attributs, etc...
— **Héritage :** Les principes sont bien respectés : abstraction, factorisation du code, etc...
— **Collections :** Les collections avec des implémentations efficaces sont utilisées.

3. Quelques **exigences non fonctionnelles** sont attendues :
   — le code rendu devra être **propre** et bien **structuré** ;
   — l'**API des classes** doit être renseignée, avec les tags `/** ... */` utilisés par `javadoc` pour générer la documentation en `html` ou `pdf`. Il n'est pas nécessaire de s'étendre plus que de raison sur une méthode type `getNbLignes()`. En revanche, la documentation devra être adaptée pour expliquer le comportement ou l'utilisation des classes ou méthodes plus avancées ;
   — assurez-vous que votre code compile et s'exécute correctement sur un **PC d'une salle machines de l'école**. Si cela ne fonctionne que dans votre environnement personnel, nous ne pourrons pas vous noter.

5.2 Rendu Final

Ce rendu se fait à la fin du module, pour évaluer les notions d'analyse et conception objet de logiciels vues en cours.

Vous indiquerez brièvement dans votre rapport quelles parties du code ont été changées lors de l'application du patron d'architecture MVC. En revanche, le code et les fonctionnalités du jeu ne seront plus évalués en détails (ceci sera noté dans le rendu intermédiaire).

Concernant une éventuelle interface graphique développée à l'aide d'une IA, l'équipe pédagogique rappelle que ce n'est pas l'objectif principal du projet. Les aspects POO et MVC restent les plus importants, et ceux qui seront évalués en priorité. L'interface graphique reste une partie en bonus. En particulier, si une équipe rend un jeu avec une interface graphique plus jolie qu'une autre équipe, due à des abonnements payants des outils IA, cela n'apportera pas plus de points d'avantage.

Annexe

A Exemple de format des données pour simuler une maison

Vous pouvez stocker une carte de la maison dans un fichier texte. Un exemple de format de description des données dans un fichier de ce type est proposé ci-dessous. Si vous utilisez des formats de ce type, il est recommandé de le mentionner dans votre rapport pour faciliter les tests sur votre programme.

```
# Un exemple de carte 8x8
8 8 10000

# l0
TERRAIN_LIBRE
FOOD
TERRAIN_LIBRE
TERRAIN_LIBRE
REVEIL
TERRAIN_LIBRE
TERRAIN_LIBRE
TRESOR
# l1
TERRAIN_LIBRE
CLÉ
TERRAIN_LIBRE
FOOD
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
# l2
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
ARME
...
...
# l3
PORTE
MUR
MUR
MUR
MUR
MUR
MUR
MUR
# l4
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
MUR
TERRAIN_LIBRE
TERRAIN_LIBRE
SORTIE
# l5
TERRAIN_LIBRE
FOOD
TERRAIN_LIBRE
TERRAIN_LIBRE
MUR
TERRAIN_LIBRE
MONSTRE
TERRAIN_LIBRE
# l6
TERRAIN_LIBRE
TERRAIN_LIBRE
...
...
TERRAIN_LIBRE
TERRAIN_LIBRE
MUR
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
# l7
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
PORTE
TERRAIN_LIBRE
TERRAIN_LIBRE
TERRAIN_LIBRE
```

FIGURE 1 – Exemple de format des données pour simuler une maison à un étage. Ici un exemple de carte 8×8, avec des cases de 10000 (mètres ?) de côté, 3 salles, 3 nourritures, 1 monstre, 1 arme, et une sortie. Le personnage se réveille en début du jeu dans la case 0/4 (ligne/colonne). Tout ce qui suit un caractère # est un commentaire, et les lignes blanches doivent être ignorées.